Hardware Layer Documentation

Educational Robot — Motor Control, Servo Actuation, Battery Monitoring & ESP32 Communication

1. Folder Purpose

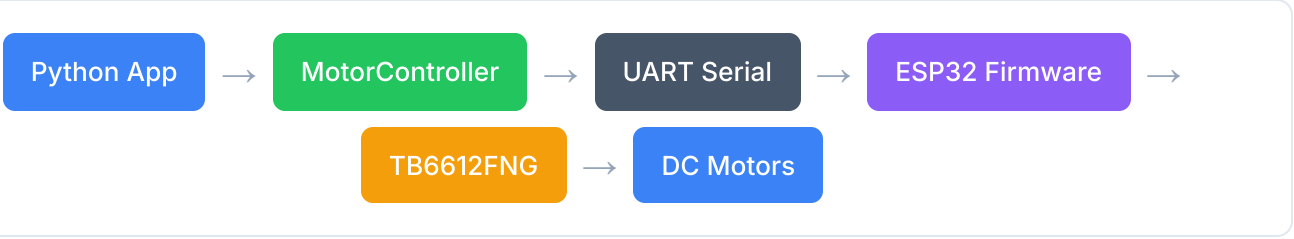
The `hardware/` package provides the complete hardware abstraction layer for the Rope educational robot. It is responsible for bridging high-level Python control logic with low-level microcontroller firmware running on an ESP32. The system supports:

- **Motor control** — TB6612FNG dual motor driver driving two DC motors (forward, backward, turn, stop, timed movement)
- **Servo control** — three servos for expressive head and arm movement
- **Battery monitoring** — voltage monitoring via ESP32 ADC with automatic low-battery shutdown
- **Serial communication** — UART-based command/response protocol between Raspberry Pi and ESP32

> **Target Microcontroller:** ESP32 DevKit V1 (DOIT) flashed via PlatformIO. The firmware uses the Arduino framework with ESP32Servo library and LEDC PWM for motor drive.

2. Architecture Overview

The hardware layer follows a **command-response pattern** over a UART serial link. The Raspberry Pi (Python process) sends ASCII commands over TX/RX to the ESP32, which interprets them and drives the physical hardware. Status and battery voltage are streamed back to the Pi.



Servo ARM_L (GPIO 21)

ESP32 Firmware → Battery ADC (GPIO 34) → Voltage Reading → BatteryMonitor

All hardware access is mediated through two Python classes: `MotorController` (serial communication with ESP32) and `BatteryMonitor` (voltage-based shutdown logic). The ESP32 firmware is a single `main.cpp` compiled with PlatformIO.

3. Files Overview

File	Lines	Role	Key Classes / Functions
`motor_controller.py`	~106	Python serial driver for ESP32	`MotorController`
`battery_monitor.py`	~101	Voltage monitoring & auto-shutdown	`BatteryMonitor`
`esp32/main.cpp`	~287	ESP32 firmware — motors, servos, battery ADC	`setup()`, `loop()`, `handle_command()`
`esp32/platformio.ini`	6	PlatformIO build configuration	—
`__init__.py`	1	Package marker	—

4. PlatformIO Configuration

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
```

```
monitor_speed = 115200
upload_speed = 921600
```

- **Board:** ESP32 DevKit (esp32dev)
- **Framework:** Arduino (uses Arduino core for ESP32)
- **Monitor speed:** 115200 baud (must match `Serial.begin()` in firmware)
- **Upload speed:** 921600 for fast flashing

The firmware depends on the `ESP32Servo` library for servo actuation. Install via PlatformIO's library manager or add to `platformio.ini`:

```
lib_deps =
  ESP32Servo
```

5. ESP32 Pin Assignment

Component	Pin	ESP32 GPIO	Notes
TB6612FNG Motor Driver			
PWMA	25	GPIO 25	Motor A PWM (LEDC channel 0, 1 kHz, 8-bit)
AIN1	26	GPIO 26	Motor A direction 1
AIN2	27	GPIO 27	Motor A direction 2
PWMB	14	GPIO 14	Motor B PWM (LEDC channel 1, 1 kHz, 8-bit)
BIN1	12	GPIO 12	Motor B direction 1
BIN2	13	GPIO 13	Motor B direction 2
STBY	33	GPIO 33	Standby (active HIGH — pulled HIGH in setup)
Servos			
SERVO_HEAD	18	GPIO 18	Head tilt servo (500–2400 µs pulse)

Component	Pin	ESP32 GPIO	Notes
SERVO_ARM_R	19	GPIO 19	Right arm servo (500–2400 µs pulse)
SERVO_ARM_L	21	GPIO 21	Left arm servo (500–2400 µs pulse)
Battery			
BATTERY_ADC	34	GPIO 34	Battery voltage divider input (0–3.3V scaled, divider ratio 3:1)
UART Serial (to Raspberry Pi)			
RX	16	GPIO 16 (UART2 RX)	Receive commands from Pi
TX	17	GPIO 17 (UART2 TX)	Send responses / battery data to Pi

> **Note:** GPIO 12 (BIN1) is a strapping pin on ESP32. Avoid pulling it HIGH during boot. The TB6612FNG uses internal pull-downs, so this is safe. If using different hardware, ensure GPIO 12 is LOW at startup.

6. Serial Command Protocol

Commands are ASCII strings sent from the Raspberry Pi (via `MotorController._send()`) to the ESP32 over Serial2 (UART2, GPIO 16/17). Each command is terminated with `\n`. The ESP32 responds on both Serial2 (back to Pi) and Serial (USB debug).

6.1 Motor Commands (Pi → ESP32)

Command	Example	Action	Response
`F`	`F\n`	Move forward continuously	`OK:F`
`F<ms>`	`F2000\n`	Move forward for <ms> milliseconds, then stop	`OK:F2000`
`B`	`B\n`	Move backward continuously	`OK:B`

Command	Example	Action	Response
`B<ms>`	`B1500\n`	Move backward for <ms> milliseconds, then stop	`OK:B1500`
`L`	`L\n`	Turn left (Motor A backward, Motor B forward)	`OK:L`
`L<ms>`	`L1000\n`	Turn left for <ms> milliseconds, then stop	`OK:L1000`
`R`	`R\n`	Turn right (Motor A forward, Motor B backward)	`OK:R`
`R<ms>`	`R800\n`	Turn right for <ms> milliseconds, then stop	`OK:R800`
`S`	`S\n`	Stop both motors immediately (sets PWM to 0)	`STOPPED`
`SPD:<n>`	`SPD:200\n`	Set motor speed (0–255). Default: 180. Constrained to [0, 255].	`OK:SPD:200`

6.2 Servo Commands (Pi → ESP32)

Command	Example	Action	Response
`HEAD:<angle>`	`HEAD:45\n`	Set head servo angle (0–180, constrained)	`OK:HEAD:45`
`ARM_R:<angle>`	`ARM_R:150\n`	Set right arm servo angle (0–180, constrained)	`OK:ARM_R:150`
`ARM_L:<angle>`	`ARM_L:30\n`	Set left arm servo angle (0–180, constrained)	`OK:ARM_L:30`
`CENTER`	`CENTER\n`	Move all servos to 90° (center position)	`OK:CENTER`

6.3 Animation Commands (Pi → ESP32)

Command	Example	Action	Response
`HAPPY`	`HAPPY\n`	Run non-blocking arm wave animation (arms swing out, ~1s, then respond)	`OK:HAPPY` (after animation completes)

6.4 Telemetry Stream (ESP32 → Pi, unsolicited)

Message	Example	Frequency	Source
`BAT:<voltage>`	`BAT:8.12`	Every 2 seconds	`loop()` — 10-sample averaged ADC reading
`STOPPED`	`STOPPED`	When timed movement expires	`loop()` — after `_move_until` elapses

6.5 Error Responses

Response	Meaning
`ERR:unknown`	Unrecognised or malformed command

7. `motor_controller.py` — MotorController

The `MotorController` class is the Python-side serial driver. It communicates with the ESP32 over a UART connection, sending ASCII commands and reading responses.

7.1 Constructor

```
MotorController(port="/dev/ttyS0", baudrate=115200, timeout=1.0)
```

- `port` — serial port device path (`COM3` on Windows, `/dev/ttyS0` on Raspberry Pi)
- `baudrate` — communication speed (must match ESP32 firmware: 115200)

- `timeout` — serial read timeout in seconds

If `pyserial` is not installed or the port cannot be opened, the controller logs a warning and sets `is_available()` to `False`. All methods gracefully return `False` when unavailable.

7.2 Public Methods

Method	Parameters	Returns	Description
`is_available()`	—	`bool`	Whether the serial port is open and ready
`forward(duration_ms=0)`	`duration_ms: int`	`bool`	Move forward (continuous if 0, timed if >0)
`backward(duration_ms=0)`	`duration_ms: int`	`bool`	Move backward (continuous if 0, timed if >0)
`turn_left(duration_ms=0)`	`duration_ms: int`	`bool`	Turn left (continuous if 0, timed if >0)
`turn_right(duration_ms=0)`	`duration_ms: int`	`bool`	Turn right (continuous if 0, timed if >0)
`stop()`	—	`bool`	Stop both motors immediately
`set_speed(speed)`	`speed: int` (0–255)	`bool`	Set motor PWM duty cycle (clamped)
`move_head(angle)`	`angle: int` (0–180)	`bool`	Set head servo angle (clamped)

Method	Parameters	Returns	Description
`move_arm_right(angle)`	`angle: int` (0–180)	`bool`	Set right arm servo angle (clamped)
`move_arm_left(angle)`	`angle: int` (0–180)	`bool`	Set left arm servo angle (clamped)
`happy()`	—	`bool`	Trigger arm wave animation
`center_servos()`	—	`bool`	Set all servos to 90°
`read_line()`	—	`Optional[str]`	Read one line from ESP32. Returns `None` if nothing available.
`close()`	—	`None`	Close the serial port

7.3 Internal Method: `_send(command)`

Encodes the command with `\n` terminator, writes to the serial port, and waits 50 ms for the ESP32 to process. Returns `True` on success, `False` if unavailable or write fails. All public motor/servo methods delegate to `_send()`.

> **Behaviour Note:** `_send()` does not wait for or validate the response from the ESP32. For timed movement commands (`F2000` , `B1500` , etc.), the ESP32 handles the stop internally using `millis()` in `loop()` , not from a Pi command.

8. `battery_monitor.py` — BatteryMonitor

The `BatteryMonitor` class runs a background thread that continuously reads battery voltage lines (`BAT:<voltage>`) sent by the ESP32 every 2 seconds. It implements a two-tier voltage threshold system with automatic Pi shutdown.

8.1 Thresholds

Voltage Range	Status	Action
> 7.2 V	Normal	No action. Resets warning countdown if previously low.
6.9 V – 7.2 V	Low	Log warning, fire `on_low_battery` callback, start 30-second shutdown countdown. If voltage recovers above 7.2 V, cancel countdown.
≤ 6.8 V	Critical	Immediate shutdown. Fire `on_shutdown` callback, execute `sudo shutdown -h now`.

8.2 Constructor

```
BatteryMonitor(motor_controller, on_low_battery=None, on_shutdown=None, on_update=None)
```

- `motor_controller` — a `MotorController` instance (used to call `read_line()`)
- `on_low_battery(voltage)` — optional callback when voltage drops to warning threshold
- `on_shutdown()` — optional callback executed before OS shutdown
- `on_update(voltage)` — optional callback on every voltage reading update

8.3 Public Methods

Method	Description
`start()`	Start the background monitoring thread. No-op if motor controller unavailable.
`stop()`	Signal the monitoring thread to stop.
`get_voltage()`	Return the latest voltage reading (thread-safe). Returns 99.0 if no reading yet.

8.4 Thread Operation (`_run()`)

1. Every 0.5 seconds, call `motor_controller.read_line()` .
2. If a line starts with `BAT:` , parse the voltage value, store it, and call `_check_voltage()` .
3. `_check_voltage()` applies the threshold logic (normal / low / critical).
4. On countdown expiry (`WARN_COUNTDOWN_SECONDS = 30`), execute `sudo shutdown -h now` via `os.system()` .

> **Shutdown Behaviour:** `_do_shutdown()` calls `os.system("sudo shutdown -h now")` . This requires the Python process to have passwordless sudo access for the `shutdown` command. Configure this in `/etc/sudoers` if not already set.

9. ESP32 Firmware (`main.cpp`) — Detailed Explanation

9.1 Setup Sequence

1. **Serial init** — `Serial.begin(115200)` for USB debug, `Serial2.begin(115200, SERIAL_8N1, 16, 17)` for UART communication with the Pi.
2. **GPIO setup** — all TB6612FNG control pins (`AIN1` , `AIN2` , `BIN1` , `BIN2` , `STBY`) set as `OUTPUT` . Battery ADC pin set as `INPUT` .
3. **PWM setup** — two LEDC channels (channel 0 for PWMA, channel 1 for PWMB), 1 kHz frequency, 8-bit resolution.
4. **STBY** — pulled HIGH to enable the TB6612FNG motor driver.
5. **Servo attach** — three servos attached to GPIOs 18, 19, 21 with 500–2400 µs pulse range. All servos set to 90°.
6. **Motors stopped** — ensure both motors are OFF at startup.
7. **Ready message** — `"ROPE Motor Controller Ready"` printed on both Serial and Serial2.

9.2 Main Loop

The `loop()` function performs three tasks on every iteration:

- **Timed movement expiry** — If `_move_until > 0` and `millis() ⩾ _move_until` , call `motors_stop()` and send `STOPPED` .
- **Battery reading** — Every 2 seconds (`BATTERY_INTERVAL`), read the ADC pin 10 times with 500 µs spacing, average, convert to voltage (`ADC / 4095.0 * 3.3 * 3.0`), and send `BAT:<voltage>` on Serial2.
- **Command processing** — If data is available on Serial2, read until `\n` , trim whitespace, and pass to `handle_command()` .

9.3 HAPPY Animation State Machine

The HAPPY animation is a non-blocking 3-phase state machine triggered by the `HAPPY` command:

Phase	Time	Action
0 → 1	Immediate (0 ms)	Set right arm to 150°, left arm to 30° (arms up and out)
1 → 2	500 ms elapsed	Return both arms to 90° (arms back to rest)
2 → Done	1000 ms elapsed	Reset flags, respond `OK:HAPPY`

The `happyActive` flag prevents re-entry. During the animation, the loop continues to check expiry, read battery, and accept new commands (new commands will move the servos out of their animation pose).

9.4 Motor Control

The TB6612FNG uses two H-bridges controlled by direction pins (`AIN1/AIN2` for Motor A, `BIN1/BIN2` for Motor B) and PWM duty cycle via LEDC:

State	AIN1	AIN2	BIN1	BIN2	Direction
Forward	HIGH	LOW	HIGH	LOW	Both motors forward
Backward	LOW	HIGH	LOW	HIGH	Both motors backward
Turn Left	LOW	HIGH	HIGH	LOW	Motor A back, Motor B forward
Turn Right	HIGH	LOW	LOW	HIGH	Motor A forward, Motor B back
Stop	LOW	LOW	LOW	LOW	Both motors coast

PWM duty cycle (0–255) is written via `ledcWrite()`. The default speed is 180. Motor power is provided by the battery; the TB6612FNG handles direction and braking via the H-bridge.

9.5 Battery Voltage Reading

The battery is read via a voltage divider on GPIO 34 (ADC channel 6):

```
sum = 0
for (int i = 0; i < 10; i++) {
```

```
    sum += analogRead(BATTERY_PIN);
    delayMicroseconds(500);
  }
  float adc = sum / 10.0;
  float vPin = (adc / ADC_MAX) * VREF;   // VREF = 3.3V, ADC_MAX = 4095
  float vBatt = vPin * DIV_RATIO;         // DIV_RATIO = 3.0
```

The divider ratio is 3:1, so a 8.4 V battery (fully charged 2S LiPo) reads as 2.8 V at the ADC pin, which maps to ~3476 ADC counts. A 10-sample average with 500 µs spacing filters noise.

> **Calibration:** The actual divider resistors (`DIV_RATIO`) must match the hardware. If using a different divider, update `DIV_RATIO` in `main.cpp` and re-flash. The Python `BatteryMonitor` thresholds (6.8 V / 7.2 V) assume a 2S LiPo chemistry (6.8 V empty, 8.4 V full).

10. Wiring Guide

Raspberry Pi	ESP32	Notes
GPIO 14 (TXD)	GPIO 16 (UART2 RX)	Pi transmits → ESP32 receives
GPIO 15 (RXD)	GPIO 17 (UART2 TX)	Pi receives ← ESP32 transmits
GND	GND	Common ground required
TB6612FNG Motor Driver (connected to ESP32)		
PWMA → GPIO 25		Motor A PWM signal
AIN1 → GPIO 26		Motor A direction 1
AIN2 → GPIO 27		Motor A direction 2
PWMB → GPIO 14		Motor B PWM signal
BIN1 → GPIO 12		Motor B direction 1
BIN2 → GPIO 13		Motor B direction 2
STBY → GPIO 33		Standby (HIGH = enabled)

Raspberry Pi	ESP32	Notes
VMOT → Battery +		Motor power supply (direct from battery)
GND → Battery –		Common ground with ESP32
Servo connections		
Head Servo → GPIO 18		Signal wire (5V power from ESP32 VIN or external BEC)
Right Arm Servo → GPIO 19		Signal wire
Left Arm Servo → GPIO 21		Signal wire
Battery monitoring		
Battery + → Voltage Divider → GPIO 34		3:1 resistive divider (e.g., 2×10 kΩ + 1×5 kΩ)
Battery – → GND		Common ground

Power Supply: The ESP32 can be powered from the Raspberry Pi's 5V pin or from a separate battery BEC. The TB6612FNG's `VMOT` must connect directly to the battery (it handles 2.5–12 V). Do not power servos from the ESP32's 3.3V rail — use a separate 5V BEC or the Pi's 5V pin if current permits.

Voltage Divider Design: For a 2S LiPo (max 8.4 V), use a 3:1 divider so that `Vout = Vin / 3`. At 8.4 V, `Vout = 2.8 V` — safe for ESP32 ADC (max 3.3 V). Use precision resistors (1% tolerance). Example: 20 kΩ + 10 kΩ (R_{top} = 20 kΩ, R_{bottom} = 10 kΩ) gives `Vout = Vin × 10 / (20 + 10) = Vin / 3`.

11. Design Decisions & Trade-offs

11.1 Why an External Microcontroller (ESP32)?

Real-time motor control (PWM generation, H-bridge sequencing, servo pulse timing) is unreliable under a Linux kernel with non-real-time scheduling. Offloading these tasks to the ESP32 ensures:

- **Deterministic timing** — motor PWM and servo refresh rates are unaffected by Linux process scheduling.
- **Fault isolation** — a Python crash does not leave motors running indefinitely (the ESP32 can implement timeout-based safety stops).
- **Power management** — the ESP32 can monitor battery voltage and initiate shutdown even if the Pi is frozen.

11.2 Text-Based Protocol vs Binary Protocol

The command protocol uses human-readable ASCII strings (e.g., `HEAD:90`) rather than a compact binary format. This is chosen for:

- **Debugging ease** — commands are directly readable from the Serial monitor.
- **Low overhead** — at 115200 baud, a 10-byte command takes ~0.87 ms to transmit. Even at 30 commands/second, serial bandwidth is negligible.
- **Firmware simplicity** — `cmd.startsWith()` is trivial compared to parsing packed binary structs.

11.3 Non-Blocking Animations

The HAPPY animation runs as a state machine inside `loop()` using `millis()` comparisons rather than `delay()`. This ensures the ESP32 can still respond to new commands, read battery, and handle timed motor expiry during the animation. Adding new animations follows the same pattern: a boolean flag, a start timestamp, and phase transitions driven by elapsed time.

11.4 Battery Monitor — Separation of Concerns

The `BatteryMonitor` Python class is intentionally independent of the `FaceModule` display system. Voltage data flows through callbacks (`on_update`) rather than direct coupling. This allows:

- The battery icon on the face display to be driven from anywhere in the application.
- Easy unit testing of shutdown logic without needing a display.
- Swapping voltage sources (e.g., I^2C fuel gauge instead of serial reading) without changing the shutdown logic.

12. Future Improvements

- **Arm gesture reactions** — add `WAVE`, `WAVE_R`, `WAVE_L` animation commands for expressive arm movement during conversations.
- **Speed ramp** — implement acceleration/deceleration ramps for smoother motion. Current speed changes are instantaneous (`ledcWrite()` is set directly).
- **Servo position feedback** — use ADC-equipped servos or potentiometers to read actual servo positions back to the Pi.
- **CAN bus for multi-ESP32 systems** — if the robot gains additional modules (e.g., sensor array, display controller), a CAN bus would provide deterministic multi-master communication.

- **Watchdog timer** — enable the ESP32 hardware watchdog to reset the microcontroller if the main loop hangs.
- **Timeout safety** — automatically stop motors if no command is received within a configurable window (e.g., 5 seconds), preventing runaway movement if the serial link drops.

> **Document Version:** 1.0 — Generated for graduation-project submission.